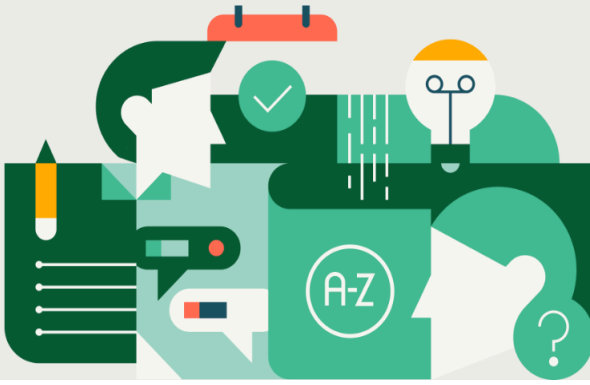
databricks

Databricks Exam Guide

Databricks Certified Associate Developer for Apache Spark



[Provide Exam Guide Feedback]

Update Information

Updates	Date
Exam Published	October 8, 2026
Exam Guide Updated	October 8, 2026

Please check the exam guide on our webpages two weeks before your exam date to ensure you are preparing for the current version of the exam.

Purpose of this Exam Guide

The purpose of this exam guide is to give you an overview of the exam and what it covers, to help you determine your exam readiness. This document is updated whenever there are changes to the exam (and indicates when those changes take effect) so that you can prepare with confidence. This guide covers the exam version that is live as of the **effective date** listed above.

Audience Description

The Databricks Certified Associate Developer for Apache Spark certification exam assesses the understanding of the Apache Spark Architecture and Components and the ability to apply the Spark DataFrame API to complete basic data manipulation tasks within a Spark session. These tasks include selecting, renaming, and manipulating columns; filtering, dropping, sorting, and aggregating rows; handling missing data; combining, reading, writing, and partitioning DataFrames with schemas; and working with UDFs and Spark SQL functions. In addition, the exam will assess the basics of the Spark architecture, like execution/deployment modes, the execution hierarchy, fault tolerance, garbage collection, lazy evaluation, Shuffling and usage of Actions and broadcasting, Structured Streaming, Spark Connect, and common troubleshooting and tuning

techniques. Individuals who pass this certification exam can be expected to complete basic Spark DataFrame tasks using Python.

About the Exam

Item	Detail
Number of scored items	45 scored multiple-choice questions
Possible unscored items	Up to 10 unscored items may be included (see "Unscored content" below)
Total number of questions you may see	Up to 55 questions (45 scored plus up to 10 unscored items)
Time limit	90 minutes (additional time is already factored in to account for unscored items)
Registration fee	USD 200, plus applicable taxes as required per local law
Delivery method	Online proctored or test center proctored
Languages offered	English
Test aides	None allowed
Prerequisite	None required; but related training highly recommended
Recommended experience	Six months of hands-on experience in Apache Spark with the listed exam outline are highly recommended
Validity	2 years

Recertification: Recertification is required every [two] years to maintain your certified status. To recertify, you must take the current live exam.

Unscored content: Exams may include unscored items to gather statistical information for future use. These items are not identified on the exam and do not affect your score, and additional time is factored into the time limit to account for them.

Recommended Preparation

Training

[Author: list current, non-deprecated training only by name. Each item must map to one or more exam sections. Remove any course that has been renamed or retired. Do NOT hyperlink courses or a learning path — Academy course names and URLs change frequently, so list course names as plain text only.]

- **Instructor-led:** Apache Spark™ Programming with Databricks
- **Self-paced (available in Databricks Academy):**
  - Introduction to Apache Spark™
  - Developing Applications with Apache Spark™
  - Stream Processing and Analysis with Apache Spark™
  - Monitoring and Optimizing Apache Spark™ Workloads on Databricks

AI Prep Guide - Any Databricks Exam

The Databricks AI Prep Guide shows you how to turn the AI chatbot you already use into a personalized tutor for any Databricks certification: prime it with the official exam guide, catch the outdated answers AI tends to give, find your weak spots, and pair it all with the hands-on practice every Databricks exam demands.

Hands-On Practice

Not sure what to practice? The AI Prep Guide (see the AI Prep Guide section above) includes a prompt under "Step 5 — Hands-on minimum" that has an AI chatbot generate a personalized checklist of 6–10 hands-on tasks mapped to this exam's objectives, each small enough to complete in Databricks Free Edition.

Databricks Documentation

As you prepare, study the official Databricks documentation for the platform areas covered on this exam. It is the most current, authoritative source. Use it to judge how deeply to study each topic and to ensure you are learning up-to-date, non-deprecated features rather than out-of-date material.
Official documentation: Databricks documentation

Exam Outline

[Author: list every section with its percentage weight. **Weights MUST sum to 100%.** Order sections as they appear on the exam blueprint. Under each section, list the objectives as they are stated on the exam. Use current product terminology only.]

Section 1: Apache Spark Architecture and Components (20%)

- Identify the advantages and challenges of implementing Spark.
- Identify the role of core components of Apache Spark™'s Architecture, including cluster, driver node, worker nodes/executors, CPU cores, and memory.
- Describe the architecture of Apache Spark™, including DataFrame and Dataset concepts, SparkSession lifecycle, caching, storage levels, and garbage collection.
- Explain the Apache Spark™ Architecture execution hierarchy.
- Configure Spark partitioning in distributed data processing, including shuffles and partitions.
- Describe the execution patterns of the Apache Spark™ engine, including actions, transformations, and lazy evaluation.
- Identify the features of the Apache Spark Modules, including Core, Spark SQL, DataFrames, Pandas API on Spark, Structured Streaming, and MLib.

Section 2: Using Spark SQL (20%)

- Utilize common data sources such as JDBC, files, etc., to efficiently read from and write to Spark DataFrames using Spark SQL, including overwriting and partitioning by column.
- Execute SQL queries directly on files, including ORC Files, JSON Files, CSV Files, Text Files, and Delta Files, and understand the different save modes for outputting data in Spark SQL.
- Save data to persistent tables while applying sorting and partitioning to optimize data retrieval.
- Register DataFrames as temporary views in Spark SQL, allowing them to be queried with SQL syntax.

Section 3: Developing Apache Spark DataFrame/DataSet API Applications (30%)

- Manipulate columns, rows, and table structures by adding, dropping, splitting, renaming column names, applying filters, and exploding arrays.
- Perform data deduplication and validation operations on DataFrames.
- Perform aggregate operations on DataFrames such as count, approximate count distinct, and mean, summary.
- Manipulate and utilize Date data type, such as Unix epoch to date string, and extract date component.

- Combine DataFrames with operations such as Inner join, left join, broadcast join, multiple keys, cross join, union, and union all.
- Manage input and output operations by writing, overwriting, and reading DataFrames with schemas.
- Perform operations on DataFrames such as sorting, iterating, printing schema, and conversion between DataFrame and sequence/list formats.
- Create and invoke user-defined functions with or without stateful operators, including StateStores.
- Describe different types of variables in Spark, including broadcast variables and accumulators.
- Describe the purpose and implementation of broadcast joins.

Section 4: Troubleshooting and Tuning Apache Spark DataFrame API Applications (10%)

- Implement performance tuning strategies & optimize cluster utilization, including partitioning, repartitioning, coalescing, identifying data skew, and reducing shuffling.
- Describe Adaptive Query Execution (AQE) and its benefits.
- Perform logging and monitoring of Spark applications - publish, customize, and analyze Driver logs and Executor logs to diagnose out-of-memory errors, cluster underutilization, etc.

Section 5: Structured Streaming (10%)

- Explain the Structured Streaming engine in Spark, including its functions, programming model, micro-batch processing, exactly-once semantics, and fault tolerance mechanisms.
- Create and write Streaming DataFrames and Streaming Datasets, including the basic output modes and output sinks.
- Perform basic operations on Streaming DataFrames and Streaming Datasets, such as selection, projection, window and aggregation.
- Perform Streaming Deduplication in Structured Streaming, both with and without watermark usage.

Section 6: Using Spark Connect to Deploy Applications (5%)

- Describe the features of Spark Connect.
- Describe the different deployment mode types (Client, Cluster, Local) in the Apache Spark™ environment.

Section 7: Using Pandas API on Spark (5%)

- Explain the advantages of using Pandas API on Spark.

- Create and invoke Pandas UDF.

Sample Questions

The purpose of sample questions is to show you the objectives as they are stated in the exam guide and to give you a sample question that aligns with each objective. The exam guide lists the objectives that could be covered on an exam; the best way to prepare is to review the full exam outline for your exam date above.

Question 1

Objective: Utilize common data sources such as JDBC, files, etc., to efficiently read from and write to Spark DataFrames using Spark SQL, including overwriting and partitioning by column.

A data engineer needs to write a DataFrame df to a Parquet file, partitioned by the column country, and overwrite any existing data at the destination path.

Which code should the data engineer use to accomplish this task in Apache Spark?

A. `df.write.mode("append").partitionBy("country").parquet("/data/output")`
B. `df.write.partitionBy("country").parquet("/data/output")`
C. `df.write.mode("overwrite").parquet("/data/output")`
D. `df.write.mode("overwrite").partitionBy("country").parquet("/data/output")`

Question 2

Objective: Perform logging and monitoring of Spark applications - publish, customize, and analyze Driver logs and Executor logs to diagnose out-of-memory errors, cluster underutilization, etc.

An engineer notices a significant increase in the job execution time during the execution of a Spark job. After some investigation, the engineer decides to check the logs produced by the Executors.

How should the engineer retrieve the Executor logs to diagnose performance issues in the Spark application?

A. Use the command 'spark-submit' with the '`--verbose`' flag to print the logs to the console.
B. Locate the executor logs on the Spark master node, typically under the /tmp directory.
C. Use the Spark UI to select the stage and view the executor logs directly from the stages tab.
D. Fetch the logs by running a Spark job with the `spark-sql` CLI tool.

Question 3

Objective: Manipulate columns, rows, and table structures by adding, dropping, splitting, renaming column names, applying filters, and exploding arrays.

Which code block will replace the `division` column in the `storesDF` DataFrame with a new column named `state`, and simultaneously replace and rename the `mName` column to `managerName` in the resulting DataFrame?

A. `storesDF = storesDF.withColumn("state", col("division")).drop("division").withColumnRenamed("mName", "managerName")`
B. `storesDF = storesDF.withColumnRenamed("state", "division").withColumnRenamed("mName", "managerName")`
C. `storesDF = storesDF.withColumn(col("division"), state).withColumnRenamed("managerName", "mName")`
D. `storesDF = storesDF.drop("division").withColumnRenamed("state", lit("default_state")).withColumnRenamed(columns={"mName": "managerName"})`

Question 4

Objective: Perform operations on DataFrames such as sorting, iterating, printing schema, and conversion between DataFrame and sequence/list formats.

Given a DataFrame `employeeDF` with columns: `name`, `department`, `salary`, and `age`.

Which code snippet sorts a DataFrame by multiple columns in descending order, prints its schema, and converts the DataFrame to a list of rows?

A. `result = employeeDF.orderBy(desc("salary"), desc("age")); employeeDF.printSchema(); row_list = result.toPandas().values.tolist()`
B. `result = employeeDF.sort("salary", "age", ascending=[False, False]); employeeDF.schema(); row_list = result.toList()`
C. `result = employeeDF.orderBy(col("salary").desc(), col("age").desc()).collect(); employeeDF.printSchema(); row_list = [row.asDict() for row in result]`
D. `result = employeeDF.sort(["salary", "age"], descending=True).show(); employeeDF.describe(); row_list = list(result)`

Question 5

Objective: Configure Spark partitioning in distributed data processing, including shuffles and partitions.

What will be the impact of setting the default value of spark.sql.shuffle.partitions to 200?

A. DataFrames will be divided into 200 distinct partitions during data shuffling operations.
B. New DataFrames created by Spark will be partitioned to utilize the memory of 200 executors optimally.
C. All DataFrames in Spark will be partitioned to occupy the memory of 200 executors perfectly.
D. Spark will only process the first 200 partitions of DataFrames to enhance performance.

Question 6

Objective: Perform data deduplication and validation operations on DataFrames.

Which code fragment will return a new DataFrame from `storesDF` that excludes all rows containing at least one missing value in any column?

A. `storesDF.na.drop("all")`
B. `storesDF.na.drop(subset = "sqft")`
C. `storesDF.na.drop()`
D. `storesDF.dropna("all")`

Question 7

Objective: Describe the different deployment mode types (Client, Cluster, Local) in the Apache Spark™ environment.

Which Spark deployment mode requires all executors to run on a single worker node?

A. Cluster mode
B. Local mode
C. Client mode
D. Standard mode

Question 8

Objective: Explain the Structured Streaming engine in Spark, including its functions, programming model, micro-batch processing, exactly-once semantics, and fault tolerance mechanisms.

A developer must calculate real-time, rolling metrics like "average session duration in the last hour" and "top 10 products viewed in the last 15 minutes" from continuous clickstream data for recommendation engines. These metrics must update every 2 minutes. They chose Streaming DataFrames over traditional batch DataFrames.

Why should the developer consider Streaming DataFrames in this use case?

A. Streaming DataFrames automatically handle data partitioning and caching more efficiently than batch DataFrames, making them faster for processing large volumes of clickstream data, even when processing historical data.

B. Streaming DataFrames enable continuous data processing with incremental updates to aggregations, allowing real-time metrics without reprocessing the entire dataset every 2 minutes.
C. Streaming DataFrames provide better error handling and automatic retry mechanisms than batch DataFrames, which is essential for handling unreliable network connections from the network recovery mechanism.
D. Streaming DataFrames use more memory than batch DataFrames because they process data in batches, making them ideal for this scenario.

Question 9

Objective: Create and invoke user-defined functions with or without stateful operators, including StateStores.

A developer is building a streaming application. The application's validation logic involves computations and string manipulations unavailable in built-in Spark functions. The developer wants to ensure the streaming job is stateless for each transaction and can scale horizontally without maintaining customer history.

When should the developer use user-defined functions (UDFs) without stateful operators for this transaction validation pipeline?

A. Maintain running totals and compare them to historical averages in state.
B. Perform windowed aggregations over the last 24 hours with incremental updates.
C. Detect session-based patterns by tracking sequences across events.
D. Apply custom logic without relying on previous transaction data or state between transaction batches.

Question 10

Objective: Perform aggregate operations on DataFrames such as count, approximate count distinct, and mean, summary.

A developer needs a dashboard for a mobile app's 50 million user activity records, showing total event and unique user counts by time. The dashboard refreshes hourly, and management prioritizes speed and accepts a 2-3% error margin for unique user counts.

Why should the developer choose `approx_count_distinct() over count(distinct())`?

A. `approx_count_distinct()` offers improved accuracy over `count(distinct())` for large datasets due to its advanced statistical algorithms.
B. `approx_count_distinct()` handles nulls and missing data better than `count(distinct())`, which is crucial for incomplete user activity logs.
C. `approx_count_distinct()` uses probabilistic algorithms `(HyperLogLog)` to boost performance by avoiding costly shuffle operations.

D. `approx_count_distinct()` uses less memory per partition than `count(distinct())` because it stores aggregated results in compressed format.

Answer Key & Explanations

[Author: every sample question must have an entry here with the correct letter, a short explanation of WHY it is correct (and, where helpful, why the common distractor is wrong), Candidates repeatedly asked for explanations, not just the letter.]

#	Answer	Explanation
1	D	Only option D applies both mode("overwrite") and partitionBy("country"), so it overwrites existing data and partitions by country. A uses append (does not overwrite), B omits the overwrite mode, and C omits the partitioning.
2	C	Executor logs are viewed through the Spark UI by selecting the stage and opening the executor logs from the stages tab. spark-submit --verbose prints configuration rather than executor logs, executor logs are not stored on a master node under /tmp, and the spark-sql CLI does not retrieve them.
3	A	Option A adds a state column from division, drops division, and renames mName to managerName, producing the required result. B, C, and D use incorrect method arguments, invalid argument order, or non-existent signatures.
4	C	Option C orders by col().desc() for descending sort, calls printSchema(), and uses collect() with asDict() to build a list of rows. The other options call non-existent methods (schema(), toList()), use invalid keywords (descending=True), or do not return rows.
5	A	spark.sql.shuffle.partitions sets the number of partitions used when shuffling data for joins and aggregations, so DataFrames are divided into 200 partitions during shuffle operations. It does not allocate executor memory or limit how many partitions are processed.
6	C	na.drop() with no arguments (how="any") removes any row containing at least one null. drop("all") and dropna("all") only drop rows where every value is null, and subset limits the check to a single column.
7	B	In Local mode all Spark processes, including executors, run in a single JVM on one node. Cluster and client modes distribute executors across worker nodes, and "Standard mode" is not a Spark deployment mode.

8	B	Streaming DataFrames process data incrementally, updating aggregations as new data arrives without reprocessing the full dataset, which suits low-latency rolling metrics refreshed every 2 minutes. The other options misdescribe how Structured Streaming works.
9	D	Stateless UDFs are appropriate when logic is applied per record without depending on prior transactions or maintained state, enabling horizontal scaling. Options A, B, and C all require state (running totals, windowed aggregations, session tracking).
10	C	approx_count_distinct() uses the HyperLogLog probabilistic algorithm to estimate distinct counts while avoiding the costly shuffle of an exact count(distinct()), trading a small error margin for speed. It is not more accurate, nor is it about null handling or compression.